ASSIGNMENT 2 – CO2

C++ Functions and Object-Oriented Programming

Name	S MOHAMED AYDIN
Register Number	192424090
Course Code	DSA0102
Assignment	Assignment 2 – CO2
Submission Date	31/10/2026
Total Marks	50

Course Outcome: Design C++ programs using functions, references, pointers, classes, arrays, static members, overloading, friend functions and inline functions.

Contents

- Q1 – Call by Value, Reference and Pointer
- Q2 – Return by Reference and Its Dangers
- Q3 – Default Arguments and Function Overloading
- Q4 – Class with an Array Inside It
- Q5 – Static Members and Arrays of Objects
- Q6 – Friend Functions and Inline Functions
- Q7 – Diagnose, Redesign and Validate
- Build, execution and GitHub submission checklist

Note: The assignment asks students to predict output before running each program and to include their own execution evidence. The predictions and expected outputs below are supplied as a guide. Run each program with your compiler, capture your actual terminal output, and add the screenshots and timing evidence before submission.

Q1 – Call by Value, by Reference and by Pointer (6 Marks)

1.1 Complete the Code: concession update

The fee is reduced by the requested percentage. Integer arithmetic is used, so any fractional part is discarded. Passing by value changes only a local copy; passing by reference changes the caller's variable; passing by pointer changes the object reached through the pointer.

```
#include <iostream>
using namespace std;

void change(int a, int& b, int* c) {
    a += 10;   // local copy only
    b += 10;   // modifies caller's b
    *c += 10;  // modifies caller's c
}

int& bigger(int& p, int& q) {
    return (p > q) ? p : q;
}

void show(int x, int y = 5, int z = 10) {
    cout << x + y + z << ' ';
}

int main() {
    int a = 1, b = 2, c = 3;
    change(a, b, &c);
    cout << a << ' ' << b << ' ' << c << '\\n';

    bigger(a, c) = 100;
    cout << a << ' ' << c << '\\n';

    show(1);
    show(1, 2);
    show(1, 2, 3);
    cout << '\\n';
}
```

1.2 Predicted output

```
1 12 13
100 13
16 13 6

1 12 13
1 100
16 13 6
```

1.3 Inquiry and design decision

• A by-value parameter is a separate local copy. A reference parameter is an alias for the caller's object. A pointer parameter stores an address; dereferencing it updates the pointed-to object.

• Use a const reference when an object must not be modified and copying it would be expensive, for example: void printStudent(const Student& s).

• For one concession, changing the fee in place through a reference is direct and efficient. If concessions must be combined, validated, or previewed, returning a new fee can be safer because the original value remains unchanged until the caller accepts the result.

1.4 Complete the Code: hostel fee

```
#include <iostream>
using namespace std;

void applyByValue(long long fee, int percent) {
    fee -= fee * percent / 100;
}

void applyConcession(long long& fee, int percent) {
    fee -= fee * percent / 100;  // TODO 1
}

int main() {
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    cout << "after call by value : " << hostelFee << '\n';
    applyConcession(hostelFee, 10);
    cout << "after call by reference: " << hostelFee << '\n';
}
```

Expected output:

```
after call by value : 45000
after call by reference: 40500
```

1.5 Alternative concept and complexity

Both reference update and return-a-new-value take O(1) time and O(1) extra space for a long long fee. A by-value call copies only one scalar, also O(1). For a large Student object, passing by value copies all its fields (and possibly owned data), while const-reference passing avoids that copy and uses O(1) parameter storage. References are best when the function must update the caller's value; returning a new value is preferable when a pure calculation or validation workflow is desired.

Q2 – Return by Reference and Its Dangers (4 Marks)

2.1 Working implementation

```
#include <iostream>
#include <vector>
using namespace std;

int& highestMark(int marks[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[best]) best = i;
    return marks[best];
}

// Never use this function: it returns a dangling reference.
int& unsafeLocalReference() {
    int local = 50;
    return local; // warning: reference to local variable
}

int main() {
    int marks[] = {78, 91, 84, 88};
    highestMark(marks, 4) += 2;
    cout << "highest after moderation: " << highestMark(marks, 4) << '\n';

    // Demonstrate the safe vector alternative using an index.
```

```
    vector<int> v = {78, 91, 84, 88};
    int best = 0;
    for (int i = 1; i < (int)v.size(); ++i)
        if (v[i] > v[best]) best = i;
    v[best] += 2;
    cout << "vector highest after moderation: " << v[best] << '\n';
}
```

Expected output:

```
highest after moderation: 93
vector highest after moderation: 93
```

2.2 Explanation

• Returning int& to an array element is valid while the array remains alive. It allows highestMark(marks, 4) += 2 because the function result is an assignable reference.

• Returning a reference to a local variable is invalid: the local is destroyed when the function returns. The reference dangles, and using it causes undefined behaviour. Compilers commonly warn about this.

• A const reference prevents modification through that reference; returning by value gives the caller an independent copy. Neither approach makes a reference to a destroyed local safe.

• A std::vector may reallocate when it grows. Reallocation invalidates pointers, references and iterators to its elements. An index can be used again after growth, but it must be checked because insertions/erasures can change element positions.

2.3 Alternative: return an index or pointer

```
int highestIndex(const int marks[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[best]) best = i;
    return best;
}
// Use: marks[highestIndex(marks, n)] += 2;
```

Finding the maximum takes O(n) time and O(1) auxiliary space whether the function returns a reference, pointer or index. An index is usually easiest to validate and can be recomputed after vector growth. A pointer/reference is convenient for direct modification but can become invalid if the underlying storage is destroyed or reallocated.

Q3 – Default Arguments and Function Overloading (6 Marks)

3.1 Why the original design is ambiguous

Declaring charge(double weight), charge(double weight, bool express), and charge(double weight, bool express=false, bool insured=false) makes a one-argument call eligible for more than one overload. Default arguments do not create separate signatures, so charge(2) can be ambiguous. The overload set should be redesigned so that each call has one best match.

3.2 Redesigned implementation

```
#include <iostream>
using namespace std;

double charge(double weightKg, bool express = false, bool insured = false) {
    double total = weightKg * 50.0;
    if (express) total += 40.0;
    if (insured) total += 25.0;
    return total;
}

double charge(int weightGrams) {
    return charge(weightGrams / 1000.0, false, false);
}

struct Parcel {
```

```
    double weightKg;
    bool express;
    bool insured;
};

double charge(const Parcel& p) {
    return charge(p.weightKg, p.express, p.insured);
}

int main() {
    cout << charge(2) << '\n';          // int overload: grams
    cout << charge(2.0f) << '\n';       // float promotes to double
    cout << charge(2.0) << '\n';        // exact double match
    cout << charge(2000) << '\n';       // int overload: grams
    cout << charge(2.0, true) << '\n';  // express, not insured
    Parcel p{2.0, true, true};
    cout << charge(p) << '\n';          // Parcel overload
}
```

Expected output:

```
0.1
100
100
0.1
140
165
```

Function selection: charge(2) chooses charge(int) by exact match; charge(2.0f) chooses charge(double, bool, bool) because float promotes to double; charge(2.0) is an exact double match; charge(2000) chooses the int overload; charge(2.0,true) chooses the double overload and uses the default insured=false; charge(p) selects the Parcel overload.

3.3 Default-argument rules and alternative design

• Overload resolution prefers an exact match, then a promotion, then a standard conversion where applicable. If two candidates are equally good, the call is ambiguous.

• Default arguments must be supplied from the right because omitted arguments form a trailing sequence. For example, (weight, express=false, insured=false) is valid; (weight, express=false, insured) is not.

• A Parcel structure makes options explicit and scales well as more delivery options are added. The simple numeric calculations are O(1) time. Passing Parcel by const reference avoids copying its fields, although this small struct is inexpensive to copy.

Design note: the example uses a clear convention: integer input means grams and floating-point input means kilograms. In a production API, distinct names such as chargeGrams() and chargeKilograms() may be clearer and safer than relying on numeric type to encode units.

Q4 – Specifying a Class with an Array Inside It (6 Marks)

4.1 SavingsAccount implementation

```
#include <iostream>
#include <string>
using namespace std;

class SavingsAccount {
private:
    string accountNumber;
    long long balance;
    long long transactions[10];
    int count;

    void record(long long amount) { // defined inside class
        if (count < 10) {
            transactions[count++] = amount;
        } else {
            for (int i = 1; i < 10; ++i)
```

```
                transactions[i - 1] = transactions[i];
            transactions[9] = amount;
        }
    }

public:
    SavingsAccount(string number, long long openingBalance)
        : accountNumber(number), balance(openingBalance), count(0) {}

    void deposit(long long amount);
    bool withdraw(long long amount);
    void miniStatement() const;
};

void SavingsAccount::deposit(long long amount) {
    if (amount <= 0) return;
    balance += amount;
    record(amount);
}

bool SavingsAccount::withdraw(long long amount) {
    if (amount <= 0 || amount > balance) return false;
    balance -= amount;
    record(-amount);
    return true;
}

void SavingsAccount::miniStatement() const {
    cout << "Account: " << accountNumber << "\nBalance: " << balance
         << "\nRecent transactions:\n";
    for (int i = 0; i < count; ++i)
        cout << transactions[i] << '\n';
}

int main() {
    SavingsAccount a("SA1001", 1000);
    for (int i = 1; i <= 11; ++i) a.deposit(i * 100);
    a.miniStatement();
}
```

4.2 Explanation and design

• The transaction array is private so external code cannot corrupt its contents or count. Public methods enforce the account rules and maintain a consistent balance and transaction history.

• When the 11th transaction arrives, shifting removes the oldest item and keeps the newest ten. This is O(k) time per new transaction, where k=10, and O(k) storage. For a fixed small k, the cost is bounded.

• A circular buffer stores a start index and writes each new transaction at the next slot. Adding is O(1), while printing the statement remains O(k). A std::deque also supports efficient insertion at the back and removal at the front.

• If the required history grows from 10 to 50, change the capacity constant or use a container. A circular buffer is efficient when the capacity is fixed; deque is more flexible when capacity varies.

The program's final balance is 7,600: opening balance 1,000 plus deposits 100+200+...+1,100 = 6,600. The mini-statement contains only the last ten deposits, from 200 through 1,100.

Q5 – Static Members and Arrays of Objects (7 Marks)

5.1 BusTicket implementation

```
#include <iostream>
using namespace std;

class BusTicket {
private:
    static int issued;
    static const int capacity = 5;
    int seatNumber;
    bool waitlisted;

public:
```

```
    BusTicket() {
        if (issued < capacity) {
            seatNumber = ++issued;
            waitlisted = false;
        } else {
            seatNumber = -1;
            waitlisted = true;
        }
    }

    void show() const {
        if (waitlisted) cout << "Waiting list\n";
        else cout << "Seat " << seatNumber << '\n';
    }

    static int seatsIssued() { return issued; }
};

int BusTicket::issued = 0;

int main() {
    BusTicket tickets[7];
    for (const auto& ticket : tickets) ticket.show();
    cout << "Seats issued: " << BusTicket::seatsIssued() << '\n';
}
```

Expected output:

```
Seat 1
Seat 2
Seat 3
Seat 4
Seat 5
Waiting list
Waiting list
Seats issued: 5
```

5.2 Explanation

• issued and capacity belong to the class as a whole, so they are static. seatNumber and waitlisted describe each individual ticket and are non-static.

• The static data member definition `int BusTicket::issued = 0;` appears outside the class in one source file. A static member function has no this pointer, so it cannot directly access a particular object's non-static members.

• An array such as `BusTicket tickets[7]` constructs seven objects in order. The shared count increases only for the five available seats; later objects are marked waitlisted.

• A global counter also uses one counter but weakens encapsulation and can conflict with other code. A separate TicketOffice object supports multiple buses independently, whereas a static member naturally shares state across every BusTicket in the process.

Complexity: constructing and displaying n tickets is O(n) time. The class-level counter uses O(1) extra storage; the ticket array uses O(n) storage for the individual objects.

Q6 – Friend Functions and Inline Functions (8 Marks)

6.1 Complete the code

```
#include <iostream>
#include <cstdlib>
using namespace std;

class Time {
    int h, m;

public:
    Time(int hh, int mm) : h(hh), m(mm) {}

    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};
```

```
inline int minutesBetween(const Time& a, const Time& b) {
    int first = a.h * 60 + a.m;
    int second = b.h * 60 + b.m;
    return abs(first - second);
}

inline bool earlier(const Time& a, const Time& b) {
    return (a.h * 60 + a.m) < (b.h * 60 + b.m);
}

int main() {
    Time departure(9, 45), arrival(13, 10);
    cout << minutesBetween(departure, arrival) << ' '
         << boolalpha << earlier(arrival, departure) << '\n';
}
```

Expected output:

```
205 false
```

6.2 Explanation

• The times are converted to minutes after midnight: 9:45 = 585 and 13:10 = 790. Their absolute difference is 205 minutes. Arrival is not earlier than departure, so earlier(arrival, departure) is false.

• A friend function is not a member function, but the class grants it access to private data. It is called like an ordinary function: minutesBetween(a,b), not a.minutesBetween(b). It is useful when an operation treats two objects symmetrically.

• Making h and m public would remove encapsulation: callers could create invalid values such as 28 hours or 90 minutes. Private data plus validated constructors/getters is safer.

• inline allows the compiler to consider replacing a call with the function body; it does not force inlining. Optimisation level, compiler decisions, and measurement noise affect timings.

6.3 Timing experiment

Compile and run the same benchmark in two builds, changing only the optimisation flag. Repeat each test several times and compare medians. Example commands: `g++ -std=c++17 -O0 time_bench.cpp -o bench_O0` and `g++ -std=c++17 -O2 time_bench.cpp -o bench_O2`. Record actual timings on the submission machine; do not invent measurements. At -O2, the compiler may inline or otherwise optimise these tiny functions, so the inline keyword may make little measurable difference.

Design decision: keep the friend functions if direct symmetric comparison is useful and the interface remains small. Alternatively, provide public getters and implement ordinary non-friend functions. Either design can be correct; the important point is to preserve encapsulation and test boundary cases (before noon, after noon, equal times).

Q7 – Diagnose, Redesign and Validate (13 Marks)

Selected program: Q4 SavingsAccount mini-statement. The original fixed-array design shifts every stored transaction whenever a new transaction arrives after the array is full. This is simple for ten entries but performs repeated moves as the capacity grows.

A. Diagnose the limitation – 2 marks

For a history capacity k, the shifting design moves k−1 values on every transaction after the buffer fills. One insertion is O(k), and m later insertions can require O(mk) assignments. The limitation is repeated movement of old transaction records, not the storage size itself.

B. Define the design objective – 2 marks

Keep the most recent k transactions in chronological display order while reducing insertion work from O(k) to O(1). Maintain O(k) storage and preserve the balance and transaction-history invariants.

C. Revised solution – 4 marks

Use a circular buffer with an array, a start index pointing at the oldest item, and a count. Until full, append at `(start + count) % capacity`. Once full, overwrite the oldest entry and advance start. This changes the location of the oldest entry rather than shifting every element.

```
class TransactionHistory {
private:
    static const int capacity = 50;
    long long values[capacity]{};
    int start = 0;    // oldest element
    int count = 0;

public:
    void add(long long amount) {
        if (count < capacity) {
            values[(start + count) % capacity] = amount;
            ++count;
        } else {
            values[start] = amount;
            start = (start + 1) % capacity;
        }
    }

    void print() const {
        for (int i = 0; i < count; ++i)
            std::cout << values[(start + i) % capacity] << '\n';
    }
};
```

D. Predict before implementing – 1 mark

Prediction: after inserting 11 transactions into a capacity-10 circular history, the history contains exactly transactions 2 through 11 in chronological order. The balance is unchanged by the choice of storage structure. Each insertion after the buffer becomes full performs one write and one index update.

E. Implement and validate – 2 marks

Test	Shifting array	Circular buffer
Insert while not full	O(1)	O(1)
Insert when full	O(k) moves	O(1)
Print k entries	O(k)	O(k)
Storage	O(k)	O(k)
Expected last 10 after 11 inserts	Items 2–11	Items 2–11

Validation procedure: compile both implementations, insert the same sequence (for example 100, 200, ..., 1100), print each history, and compare the output. Both must show 200 through 1100 for capacity 10. Measure a large number of insertions at capacities 10, 50, 500 and 5,000, using the same compiler and optimisation flags. Record the observed values in the table below after running locally.

Capacity k	Shifting time (ms)	Circular time (ms)	Environment / flags
10	Fill after run	Fill after run	g++ -std=c++17 -O2
50	Fill after run	Fill after run	g++ -std=c++17 -O2
500	Fill after run	Fill after run	g++ -std=c++17 -O2
5000	Fill after run	Fill after run	g++ -std=c++17 -O2

F. Defend the final design – 2 marks

The circular buffer meets the objective because insertion remains O(1) even when full, while memory remains O(k). Its main trade-off is slightly more index arithmetic and a less obvious internal representation. A shifting array may be preferable for very small histories when simplicity matters more

than insertion cost. For a history that grows dynamically, std::deque is a strong alternative; its interface is simpler, but it may allocate memory dynamically.

Final Summary and Submission Checklist

• Create a GitHub repository named `DSA01-192421373-Assignment2` (adjust only if the faculty specifies a different exact pattern).

• Create one folder per task (Task01 through Task07) and save the relevant .cpp source files, any input files, and actual output screenshots.

• Add README.md with name, register number, course code, compiler/version, exact compile commands, run steps, and a short result note for each task.

• Commit progressively: at least one meaningful commit per task. Do not upload everything in a single final commit.

• Before submission, compile every program with warnings enabled, e.g. `g++ -std=c++17 -Wall -Wextra file.cpp -o program`, and record compiler warnings where required.

• Replace the timing placeholders in Q6 and Q7 with measurements from your own computer. Add your GitHub link to this report's cover page or the designated submission field.

Academic integrity reminder: this report gives implementations, predictions and analysis based on the supplied assignment screenshots. You must execute the programs yourself, include genuine output screenshots and timing results, and be prepared to explain the code during the verification/viva.